

Relatório Técnico — Sistema de Gerenciamento Hotel do Mosquito

Disciplina: Laboratório de Banco de Dados

Entrega: 01/06/2026

Integrantes:

- Camili de Moura Marangoni
 - Lucas Sobrinho Santos
 - Maria Eduarda Patu Ângelo da Silva
 - Matheus Pinheiro de Camargo Silva
 - Willian Alexandre Schwingel Ferreira
-

Sumário

1. [Resumo do sistema](#)
 2. [Modelagem de dados](#)
 3. [Encapsulamento total via Stored Procedures](#)
 4. [Views](#)
 5. [Stored Procedures transacionais](#)
 6. [Transações puras](#)
 7. [Triggers](#)
 8. [Segurança](#)
 9. [Interface gráfica](#)
 10. [Casos de teste](#)
 11. [Principais dificuldades encontradas](#)
-

1. Resumo do sistema

O **Hotel do Mosquito** é um sistema de gerenciamento hoteleiro desenvolvido como trabalho final da disciplina de Laboratório de Banco de Dados. O sistema cobre o ciclo completo de uma estadia: cadastro de clientes e quartos, registro de reservas, realização de check-in, lançamento de consumos durante a hospedagem e encerramento via check-out com cálculo automático do valor total.

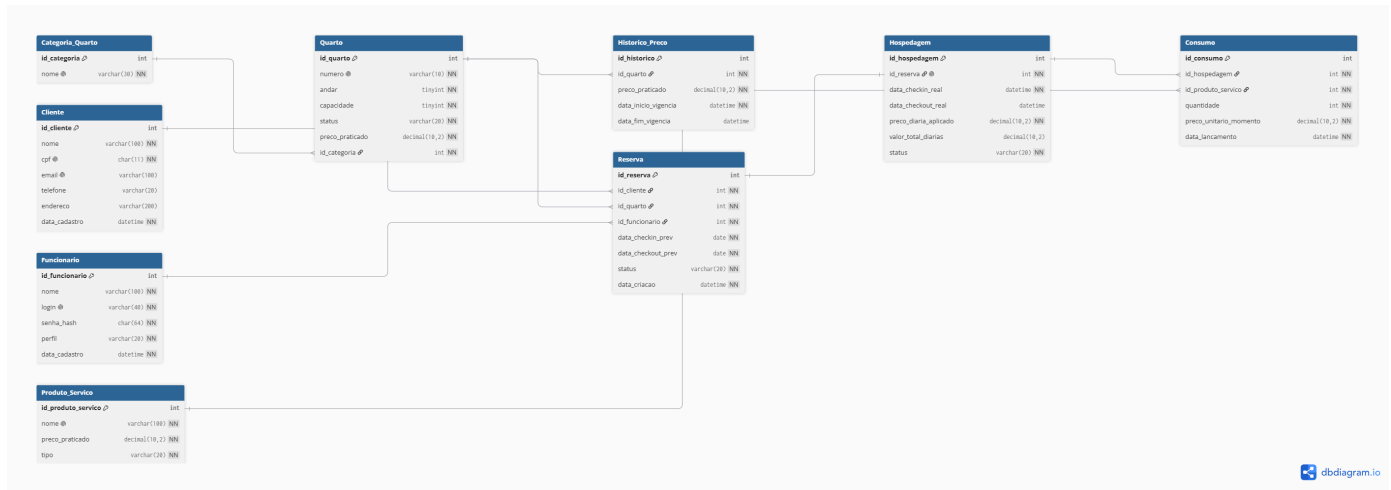
A diretriz central do projeto é o **encapsulamento total**: toda interação com o banco de dados ocorre exclusivamente por meio de stored procedures ou views. A aplicação Python nunca emite instruções SQL brutas — nenhum **INSERT**, **UPDATE**, **DELETE** ou **SELECT** em tabela é enviado diretamente pela rede. Isso garante que as regras de negócio residam no banco de dados e não na aplicação, tornando o sistema robusto contra adulterações, injeção de SQL e inconsistências causadas por clientes alternativos (scripts, ferramentas de administração).

Stack tecnológico:

Componente	Tecnologia
Banco de dados	MySQL 8.0 · InnoDB · utf8mb4
Driver de conexão	mysql-connector-python 8.4.0
Interface gráfica	Python 3.10+ · Tkinter / ttk
Seletor de data	tkcalendar 1.6.1
Infraestrutura local	Docker Compose (MySQL 8 em container)

2. Modelagem de dados

2.1 Diagrama Entidade-Relacionamento (MER)



2.2 Modelo Lógico Relacional

O banco é composto por **9 tabelas**:

```

Categoria_Quarto (id_categoria PK, nome UNIQUE)

Quarto (id_quarto PK, numero UNIQUE, andar, capacidade,
        status ENUM, preco_praticado CHECK≥0,
        id_categoria FK→Categoria_Quarto)

Historico_Precos (id_historico PK, id_quarto FK→Quarto,
                 preco_praticado, data_inicio_vigencia,
                 data_fim_vigencia) -- NULL = vigente

Cliente (id_cliente PK, nome, cpf UNIQUE, email UNIQUE,
         telefone, endereco, data_cadastro)

Funcionario (id_funcionario PK, nome, login UNIQUE,
             senha_hash CHAR(64), perfil ENUM,
             data_cadastro)

Reserva (id_reserva PK, id_cliente FK, id_quarto FK,
         id_funcionario FK, data_checkin_prev,
         data_checkout_prev CHECK>checkin,
         status ENUM DEFAULT 'Confirmada', data_criacao)

Hospedagem (id_hospedagem PK, id_reserva FK UNIQUE,
            data_checkin_real, data_checkout_real,
            preco_diaria_aplicado, valor_total_diarias,
            status ENUM DEFAULT 'Ativa')

Produto_Servico (id_produto_servico PK, nome UNIQUE,
                preco_praticado CHECK≥0, tipo ENUM)

Consumo (id_consumo PK, id_hospedagem FK→Hospedagem CASCADE,
         id_produto_servico FK, quantidade CHECK>0,
         preco_unitario_momento, data_lancamento)
  
```

2.3 Justificativa da normalização

1FN — Primeira Forma Normal: todos os atributos são atômicos e de domínio simples. O campo **endereco** de **Cliente** foi mantido como **VARCHAR** único por decisão de projeto consciente, alinhada ao enunciado da disciplina; uma decomposição em Logradouro/Número/Cidade/UF/CEP seria tecnicamente mais correta, mas não agrega valor ao escopo do trabalho.

2FN — Segunda Forma Normal: todas as chaves primárias são simples (**id_*** inteiros auto-incrementais), portanto não há como existir dependência parcial. O modelo está em 2FN por construção.

3FN — Terceira Forma Normal: o modelo não apresenta dependências transitivas. Cada atributo depende diretamente e exclusivamente de sua chave primária:

- `Categoria_Quarto` está isolada de `Quarto` — o nome da categoria não é atributo derivado do quarto.
- `Historico_Preco` separa o estado histórico do estado atual de `Quarto.preco_praticado`.
- `Consumo.preco_unitario_momento` e `Hospedagem.preco_diaria_aplicado` são **snapshots transacionais** — não são dependências transitivas, mas registros deliberados do valor no momento do evento, garantindo imutabilidade do faturamento histórico. Este padrão é amplamente utilizado em sistemas transacionais de produção e é plenamente compatível com a 3FN.

2.4 Decisões de modelagem relevantes

Decisão	Justificativa
<code>Quarto.preco_praticado</code> como coluna explícita	Permite que o trigger observe mudanças de preço e mantenha o histórico automaticamente
<code>Hospedagem.preco_diaria_aplicado</code>	Snapshot do preço no check-in; reajustes futuros não afetam estadias em curso
<code>Consumo.preco_unitario_momento</code>	Snapshot do preço do produto no lançamento; garante que alterações de preço não reescrevam o faturamento passado
<code>Reserva.status DEFAULT 'Confirmada'</code>	Toda reserva nasce confirmada; não existe etapa intermediária de "confirmação manual"
<code>Hospedagem.id_reserva UNIQUE</code>	Relação 1:1 estrita — uma reserva gera no máximo uma hospedagem
<code>Consumo → Hospedagem ON DELETE CASCADE</code>	Ao excluir uma hospedagem (operação administrativa), seus consumos são removidos em cascata; todas as demais FKs usam RESTRICT

2.5 Índices adicionais

Além dos índices implícitos de PK e FK, foram criados índices compostos para otimizar as operações mais frequentes:

Índice	Tabela	Operação beneficiada
<code>(id_quarto, data_checkin_prev, data_checkout_prev)</code>	Reserva	Deteção de sobreposição em <code>sp_RegistrarReserva</code>
<code>(id_cliente, status)</code>	Reserva	Histórico de reservas por cliente
<code>(id_hospedagem)</code>	Consumo	Soma de consumos no check-out
<code>(id_quarto, data_fim_vigencia)</code>	Historico_Preco	Consulta do preço em vigor em data específica

3. Encapsulamento total via Stored Procedures

3.1 Princípio

A aplicação Python possui um único mecanismo de comunicação com o banco: a função `call_proc()`, que usa `cursor.callproc()` do driver oficial. Nenhuma string SQL é construída, interpolada ou executada diretamente. O acesso a views é feito exclusivamente por `SELECT * FROM vw_nome`, sem cláusulas `WHERE` dinâmicas com concatenação.

```
# app/db/connection.py – única forma de escrever no banco
def call_proc(conn, proc_name, params=()):
    cursor = conn.cursor()
    cursor.callproc(proc_name, tuple(params))
    results = [rs.fetchall() for rs in cursor.stored_results()]
    return results

# app/db/connection.py – única forma de ler views
def query_view(conn, view_name, where_clause="", params=()):
    sql = f"SELECT * FROM {view_name} {where_clause}".strip()
    cursor = conn.cursor(dictionary=True)
    cursor.execute(sql, tuple(params))
    return cursor.fetchall()
```

O usuário MySQL utilizado pela aplicação recebe apenas **EXECUTE** em procedures e **SELECT** em views — sem acesso direto a tabelas.

3.2 Mapa completo de Stored Procedures (~26 no total)

CRUDs básicos (16 procedures):

Entidade	Create	Read	Update	Delete
Cliente	<code>sp_Cliente_Create</code>	<code>sp_Cliente_Read</code>	<code>sp_Cliente_Update</code>	<code>sp_Cliente_Delete</code>
Quarto	<code>sp_Quarto_Create</code>	<code>sp_Quarto_Read</code>	<code>sp_Quarto_Update</code>	<code>sp_Quarto_Delete</code>
Funcionário	<code>sp_Funcionario_Create</code>	<code>sp_Funcionario_Read</code>	<code>sp_Funcionario_Update</code>	<code>sp_Funcionario_Delete</code>
Reserva	<code>sp_RegistrarReserva</code>	<code>sp_Reserva_Read</code>	<code>sp_Reserva_Update</code>	<code>sp_Reserva_Delete</code>
Categoria	<code>sp_Categoria_Create</code>	<code>sp_Categoria_Read</code>	<code>sp_Categoria_Update</code>	<code>sp_Categoria_Delete</code>
Produto/Serviço	<code>sp_ProdutoServico_Create</code>	<code>sp_ProdutoServico_Read</code>	<code>sp_ProdutoServico_Update</code>	<code>sp_ProdutoServico_Delete</code>

Procedures especializadas:

Procedure	Finalidade
<code>sp_Login</code>	Autenticação com SHA2; nunca retorna hash
<code>sp_AssertGerente</code>	Helper interno — lança SIGNAL se executor não for Gerente
<code>sp_AtualizarPrecoQuarto</code>	Única que modifica <code>preco_praticado</code> ; requer Gerente; dispara trigger
<code>sp_AtualizarCategoriaQuarto</code>	Altera categoria do quarto; requer Gerente
<code>sp_HistoricoPreco_ReadPorQuarto</code>	Consulta linha do tempo de preços de um quarto
<code>sp_RealizarCheckIn</code>	Transacional — valida reserva, atualiza status, cria Hospedagem com snapshot
<code>sp_RealizarCheckOut</code>	Transacional — calcula total, finaliza Hospedagem, libera quarto
<code>sp_LancarConsumo</code>	Transacional — registra item consumido com snapshot de preço
<code>sp_Relatorio_TaxaOcupacao</code>	Relatório gerencial (Gerente only)
<code>sp_Relatorio_TopClientes</code>	Relatório gerencial (Gerente only)
<code>sp_Relatorio_TopQuartos</code>	Relatório gerencial (Gerente only)
<code>sp_Relatorio_FaturamentoMensal</code>	Relatório gerencial (Gerente only)

4. Views

4.1 Visão geral

As 7 views eliminam a necessidade de JOINS complexos na aplicação. A camada Python simplesmente lê `SELECT * FROM vw_nome` e recebe dados prontos para exibição.

4.2 Views obrigatórias

`vw_quartos_disponiveis`

Retorna quartos com `status = 'Disponivel'` junto ao nome da categoria e preço praticado. Utilizada para popular o combobox de quarto no cadastro de reservas.

`vw_ocupacao_atual`

Lista todas as hospedagens com `status = 'Ativa'`, incluindo nome do cliente, número do quarto, data de check-in real, preço da diária aplicado e soma dos consumos lançados até o momento. Alimenta a tela de Hospedagens Ativas.

`vw_historico_reservas_cliente`

Exibe o histórico completo de reservas com dados do cliente, quarto, funcionário e datas. Consultável por CPF do cliente.

4.3 Views de relatório gerencial

`vw_taxa_ocupacao_mensal`

Calcula o percentual de ocupação por mês, relacionando hospedagens finalizadas ao total de quartos disponíveis.

`vw_top_clientes_assiduos`

Agrega o número total de diárias por cliente, ordenado de forma decrescente. Identifica os clientes mais frequentes.

`vw_top_quartos_reservados`

Conta o número de reservas por quarto, ordenado de forma decrescente. Identifica os quartos com maior demanda.

vw_faturamento_mensal

Soma `valor_total_diarias` + `total de consumos` por mês de check-out, fornecendo a receita mensal do hotel.

5. Stored Procedures transacionais

Quatro procedures encapsulam operações que envolvem múltiplas tabelas e devem ser atômicas. Todas seguem o mesmo padrão:

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    ROLLBACK; -- desfaz tudo em caso de qualquer erro
    RESIGNAL; -- repassa a mensagem de erro ao chamador
END;

START TRANSACTION;
-- validações de negócio (podem lançar SIGNAL)
-- operações DML em múltiplas tabelas
COMMIT;
```

O `EXIT HANDLER` garante que, se qualquer instrução falhar (inclusive um `SIGNAL` de validação de negócio), a transação inteira é revertida. O `RESIGNAL` repassa a mensagem original de erro para o driver Python, que a apresenta ao usuário como `RuntimeError`.

5.1 sp_RegistrarReserva

Registra uma nova reserva, verificando antes se o quarto está disponível para o período solicitado.

Lógica de detecção de sobreposição:

```
SELECT COUNT(*) INTO v_conflitos
FROM Reserva
WHERE id_quarto = p_id_quarto
AND status IN ('Confirmada', 'Efetivada')
AND NOT (
    data_checkout_prev <= p_data_checkin_prev
    OR data_checkin_prev >= p_data_checkout_prev
);
```

A condição `NOT (fim_existente ≤ inicio_novo OR inicio_existente ≥ fim_novo)` é a forma canônica de detecção de sobreposição de intervalos. Se qualquer reserva ativa do mesmo quarto tiver período que se intersecta com o solicitado, `v_conflitos > 0` e um `SIGNAL` é lançado.

5.2 sp_RealizarCheckIn

Realiza o check-in de uma reserva confirmada.

Validações:

- Reserva deve ter `status = 'Confirmada'`
- Quarto deve ter `status IN ('Disponivel', 'Limpeza')`

Operações (atômicas):

1. `UPDATE Reserva SET status = 'Efetivada'`
2. `UPDATE Quarto SET status = 'Ocupado'`
3. `INSERT INTO Hospedagem (preco_diaria_aplicado = Quarto.preco_praticado)` — o preço é capturado neste momento exato (snapshot), garantindo que reajustes futuros não alterem o valor a ser cobrado da hospedagem em curso.

A procedure retorna o `id_hospedagem` gerado para uso imediato pela aplicação.

5.3 sp_RealizarCheckOut

Encerra a hospedagem e calcula o valor total a ser cobrado.

Fórmula de cálculo:

```
valor_total = (nº de diárias × preco_diaria_aplicado)
              + SUM(Consumo.quantidade × Consumo.preco_unitario_momento)
```

O número de diárias é calculado como `DATEDIFF(NOW(), data_checkin_real)`, com mínimo de 1 dia.

Operações (atômicas):

1. `UPDATE Hospedagem SET valor_total_diarias, data_checkout_real = NOW(), status = 'Finalizada'`
2. `UPDATE Quarto SET status = 'Limpeza'`

A procedure retorna um resultset com o resumo financeiro da estadia (quantidade de diárias, preço unitário, total de diárias, total de consumos, valor total), exibido ao recepcionista na tela de check-out.

5.4 `sp_LancarConsumo`

Registra um item consumido pelo hóspede durante a hospedagem.

Validações:

- Hospedagem deve ter `status = 'Ativa'`
- `quantidade` deve ser positiva

Operação:

```
INSERT INTO Consumo (
  id_hospedagem, id_produto_servico, quantidade,
  preco_unitario_momento, -- snapshot do preco_praticado atual do produto
  data_lancamento
) VALUES (... , v_preco_atual, NOW());
```

O snapshot do preço do produto no momento do lançamento garante que eventuais reajustes de preço de cardápio não alterem o faturamento de consumos já registrados.

6. Transações puras

6.1 Seed transacional — `sql/05_seed_data.sql`

A carga inicial de dados de exemplo é realizada dentro de uma única transação, com `SAVEPOINT` entre cada grupo de entidades. Isso garante que o banco sempre contenha um conjunto coerente e completo de dados — se qualquer inserção falhar, é possível reverter apenas a seção problemática sem perder os dados já inseridos.

```
START TRANSACTION;
INSERT INTO Categoria_Quarto ...;
SAVEPOINT sp_categorias;

INSERT INTO Quarto ...;           -- trigger cria Historico_Preco automaticamente
SAVEPOINT sp_quartos;

INSERT INTO Cliente ...;
SAVEPOINT sp_clientes;

INSERT INTO Funcionario ...;     -- senhas inseridas via SHA2(senha, 256)
SAVEPOINT sp_funcionarios;

INSERT INTO Produto_Servico ...;
SAVEPOINT sp_produtos;

INSERT INTO Reserva ...;
SAVEPOINT sp_reservas;

INSERT INTO Hospedagem ...;
SAVEPOINT sp_hospedagens;
```

```
INSERT INTO Consumo ...;  
COMMIT;
```

Resultado após o seed: **5** categorias · **15** quartos · **10** clientes · **5** funcionários · **10** produtos/serviços · **10** reservas · **8** hospedagens · **15** consumos.

6.2 Script demonstrativo — `sql/99_demo_transacoes.sql`

O script demonstra os três mecanismos de controle transacional exigidos pela disciplina:

Cenário 1 — COMMIT normal:

Atualiza o preço de um quarto → trigger registra nova vigência em `Historico_Preco` → `COMMIT`. O banco fica no estado atualizado. Demonstra que trigger e transação coexistem corretamente.

Cenário 2 — ROLLBACK explícito:

Inicia transação → atualiza preço (trigger cria histórico) → exibe estado intermediário → `ROLLBACK`. O banco retorna ao estado original. Demonstra que o `ROLLBACK` desfaz inclusive os efeitos do trigger, pois este executa dentro da mesma transação.

Cenário 3 — SAVEPOINT com rollback parcial:

Atualiza dois quartos com `SAVEPOINT` entre as operações → `ROLLBACK TO SAVEPOINT` → apenas a segunda alteração é desfeita. Demonstra controle granular de transações.

7. Triggers

7.1 Justificativa do uso de triggers

A invariante "*toda alteração no preço de um quarto deve gerar um registro em `Historico_Preco`*" foi implementada como trigger, e não como lógica da stored procedure, por uma razão fundamental: **o trigger é executado pelo SGBD independentemente do cliente que disparou a operação**. Mesmo que um administrador altere o preço diretamente via MySQL Workbench, script de migração ou qualquer outro cliente, o histórico será mantido automaticamente. Esta é a utilização canônica de triggers em sistemas de produção: proteger invariantes que devem ser absolutas.

7.2 `trg_preco_inicial_quarto` — AFTER INSERT ON Quarto

Criado para garantir que todo quarto nasça com um registro histórico de preço.

```
CREATE TRIGGER trg_preco_inicial_quarto  
AFTER INSERT ON Quarto  
FOR EACH ROW  
BEGIN  
    INSERT INTO Historico_Preco (  
        id_quarto,  
        preco_praticado,  
        data_inicio_vigencia,  
        data_fim_vigencia  
    ) VALUES (  
        NEW.id_quarto,  
        NEW.preco_praticado,  
        NOW(),  
        NULL -- NULL indica vigência em aberto (preço atual)  
    );  
END;
```

Efeito: ao inserir um quarto via `sp_Quarto_Create`, o banco automaticamente registra o primeiro ponto da linha do tempo de preços, com `data_fim_vigencia = NULL` indicando que esse preço está atualmente em vigor.

7.3 `trg_atualiza_preco_quarto` — AFTER UPDATE ON Quarto

Mantém a linha do tempo de preços sempre atualizada quando o preço de um quarto é modificado.

```
CREATE TRIGGER trg_atualiza_preco_quarto  
AFTER UPDATE ON Quarto  
FOR EACH ROW  
BEGIN
```

```

IF OLD.preco_praticado <> NEW.preco_praticado THEN

    -- Fecha a vigência do preço anterior
    UPDATE Historico_Precos
    SET data_fim_vigencia = NOW()
    WHERE id_quarto = NEW.id_quarto
        AND data_fim_vigencia IS NULL;

    -- Abre nova vigência com o preço atual
    INSERT INTO Historico_Precos (
        id_quarto,
        preco_praticado,
        data_inicio_vigencia,
        data_fim_vigencia
    ) VALUES (
        NEW.id_quarto,
        NEW.preco_praticado,
        NOW(),
        NULL
    );

END IF;
END;

```

A condição `IF OLD.preco_praticado <> NEW.preco_praticado` evita que atualizações em outros campos do quarto (como `status`) criem entradas espúrias no histórico.

Invariante garantida: em qualquer momento, existe exatamente **uma** linha em `Historico_Precos` com `id_quarto = X` e `data_fim_vigencia IS NULL`, representando o preço atual. Toda linha com `data_fim_vigencia NOT NULL` representa um preço histórico encerrado.

8. Segurança

8.1 Armazenamento seguro de senhas

As senhas dos funcionários são armazenadas como `CHAR(64)` contendo o resultado de `SHA2(senha, 256)` — um hash hexadecimal de 64 caracteres. O hash é computado **dentro do servidor MySQL**, nas procedures `sp_Funcionario_Create`, `sp_Funcionario_Update` e `sp_Login`. A senha em texto puro percorre a rede do cliente ao servidor, mas nunca é persistida nem retornada.

```

-- sp_Login: compara hash, nunca retorna senha
SELECT id_funcionario, nome, perfil
FROM Funcionario
WHERE login = p_login
    AND senha_hash = SHA2(p_senha, 256);

-- sp_Funcionario_Create: armazena apenas o hash
INSERT INTO Funcionario (login, senha_hash, perfil, ...)
VALUES (p_login, SHA2(p_senha, 256), p_perfil, ...);

```

Nota acadêmica: `SHA2` sem salt é adequado para o escopo deste trabalho. Em ambiente de produção, recomenda-se `bcrypt` ou `argon2` com salt por usuário, implementados na camada de aplicação, pois esses algoritmos são resistentes a ataques de dicionário e rainbow tables.

8.2 Controle de acesso por perfil (RBAC)

O sistema implementa controle de acesso em duas camadas independentes, seguindo o princípio de **defesa em profundidade**:

Camada 1 — Banco de dados (`sp_AssertGerente`):

```

CREATE PROCEDURE sp_AssertGerente(IN p_id_executor INT)
BEGIN
    DECLARE v_perfil ENUM('Recepcionista', 'Gerente');
    SELECT perfil INTO v_perfil
    FROM Funcionario WHERE id_funcionario = p_id_executor;

```



```

IF v_perfil IS NULL THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Executor nao encontrado';
END IF;
IF v_perfil <> 'Gerente' THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Acesso negado: operacao restrita a Gerentes';
END IF;
END;

```

Este helper é chamado no início de todas as procedures sensíveis:

- `sp_Funcionario_Create/Update/Delete`
- `sp_AtualizarPrecoQuarto`
- `sp_AtualizarCategoriaQuarto`
- `sp_Relatorio_TaxaOcupacao, _TopClientes, _TopQuartos, _FaturamentoMensal`

Mesmo que um Recepcionista obtenha as credenciais do banco e tente chamar essas procedures diretamente via Workbench, o bloqueio é aplicado.

Camada 2 — Interface gráfica:

Recurso	Recepcionista	Gerente
Aba Funcionários	Não exibida	Exibida
Aba Relatórios	Não exibida	Exibida
Botão "Alterar Preço" (Quartos)	Desabilitado	Habilitado
Combobox de Categoria ao editar quarto	Desabilitado	Habilitado
Formulário de edição de quartos	Não exibido	Exibido

A camada de UI oferece conveniência e clareza; a camada de banco oferece garantia real.

8.3 Prevenção de SQL Injection

O sistema é imune a SQL Injection por design, por três razões complementares:

1. **Ausência de SQL dinâmico:** a aplicação nunca constrói strings SQL em tempo de execução. Os únicos "SQLs" enviados são chamadas `CALL nome_procedure(?, ?, ?)` com placeholders.
2. **Parâmetros vinculados pelo driver:** `cursor.callproc("sp_Nome", (valor1, valor2))` envia os valores como dados tipados, separados do código SQL. O MySQL os trata como literais, nunca como código.
3. **Permissões mínimas:** o usuário da aplicação tem apenas `EXECUTE` em procedures e `SELECT` em views. Mesmo que um atacante injetasse SQL com sucesso (o que é impossível pelo ponto 2), não teria permissão para executar DDL, DML direto ou acessar tabelas sensíveis.

9. Interface gráfica

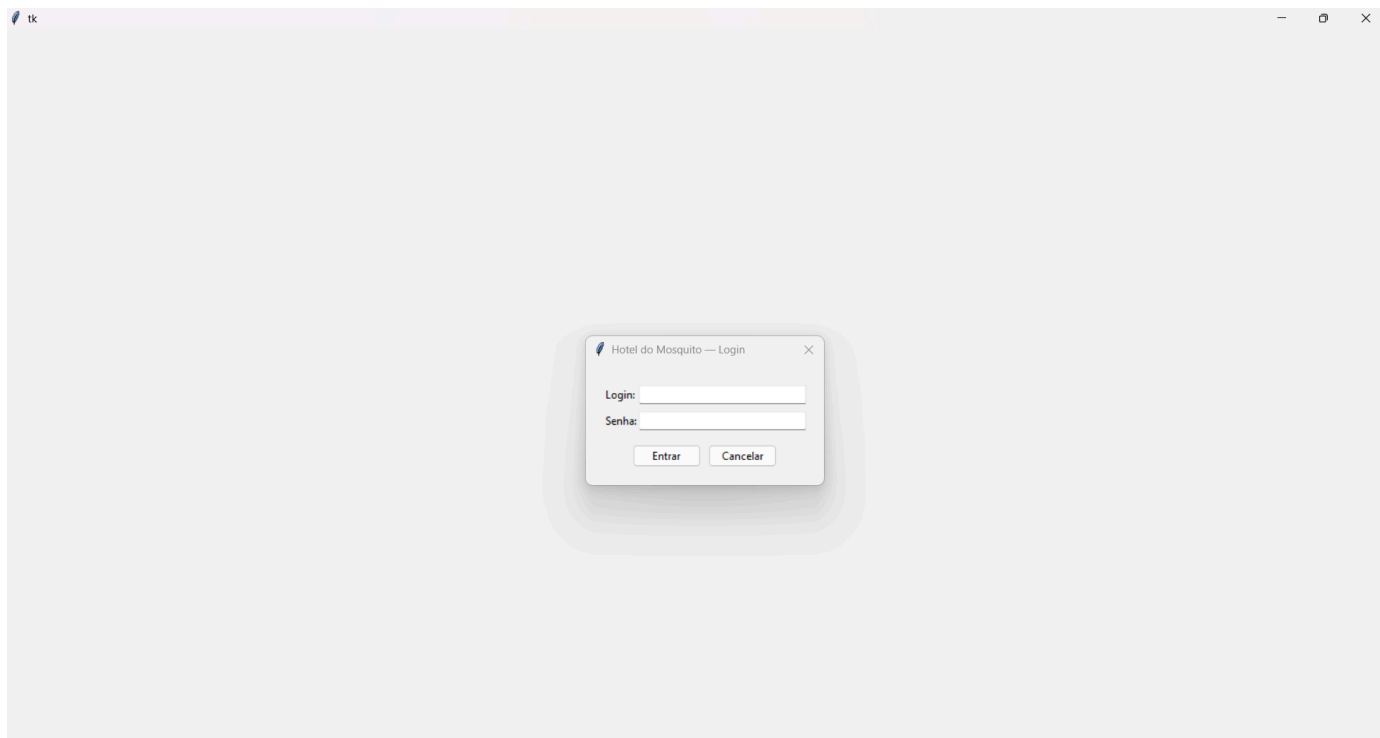
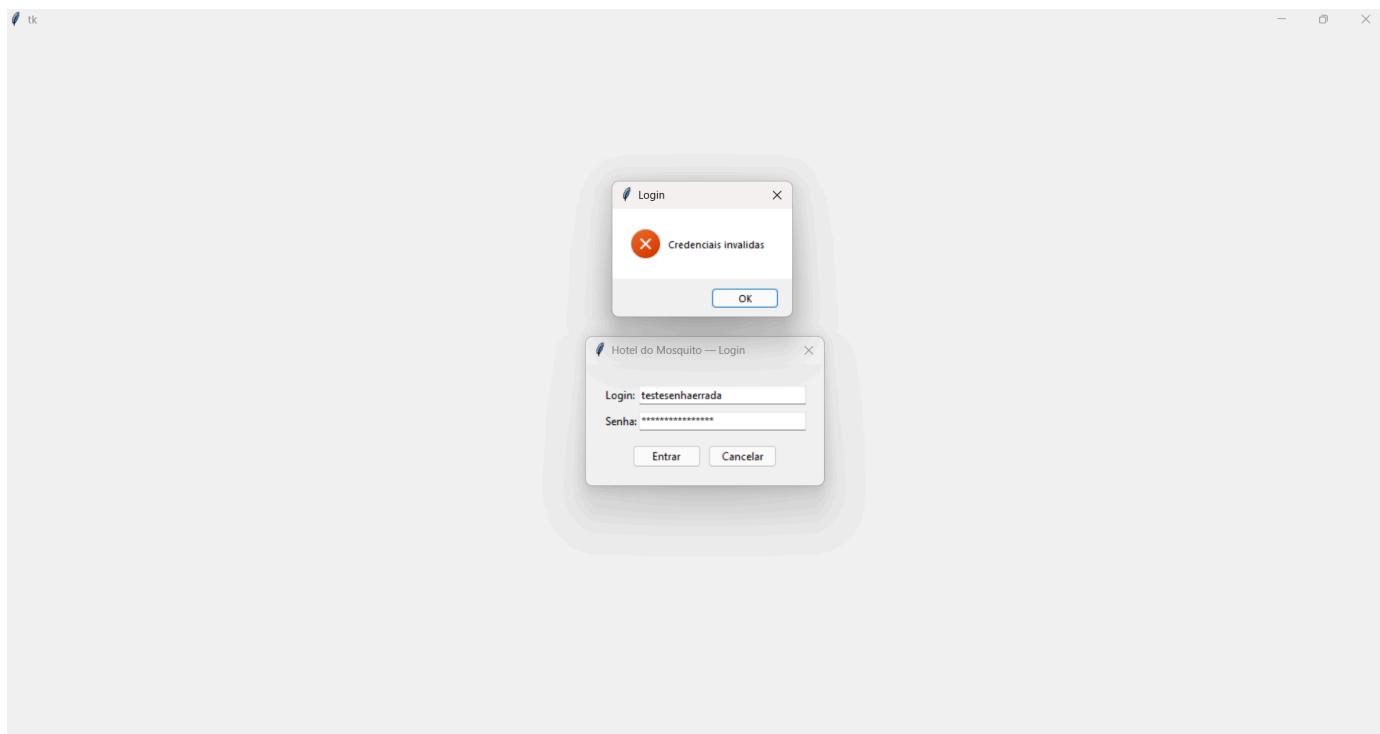
A interface foi desenvolvida com **Python 3 + Tkinter (ttk.Notebook)**, seguindo o princípio de que a UI existe para demonstrar as funcionalidades do banco, não para conter lógica de negócio.

Funcionalidades por perfil:

Aba	Recepcionista	Gerente
Clientes	Somente leitura	CRUD completo
Quartos	Somente leitura (tabela)	CRUD + Alterar Preço + Ver Histórico
Funcionários	—	CRUD completo
Reservas	Registrar + Cancelar + Calendário de datas	Idem
Hospedagem	Check-in + Consumo + Check-out	Idem
Relatórios	—	4 relatórios gerenciais

Decisões de UX relevantes:

- A aba inicial do Recepcionista é **Hospedagem** (operação principal do dia a dia).
- A tela de Hospedagem é uma **vista única** sem sub-abas: reservas agendadas aguardando check-in no topo e hospedagens ativas com consumo e check-out na parte inferior. Elimina a necessidade de trocar de aba para realizar o fluxo completo.
- Ao trocar de aba, os dados são recarregados automaticamente (<<NotebookTabChanged>>), garantindo que o usuário sempre veja informações atualizadas.
- Campos de data utilizam **DateEntry** (tkcalendar) com seletor de calendário visual.
- A janela abre maximizada com a tela de login centralizada, tornando imediato para o usuário que é necessário autenticar-se.

Figura 1 — Tela de login**Figura 2 — Login com credencial inválida****Figura 3 — Tela principal (perfil Gerente — 6 abas)**

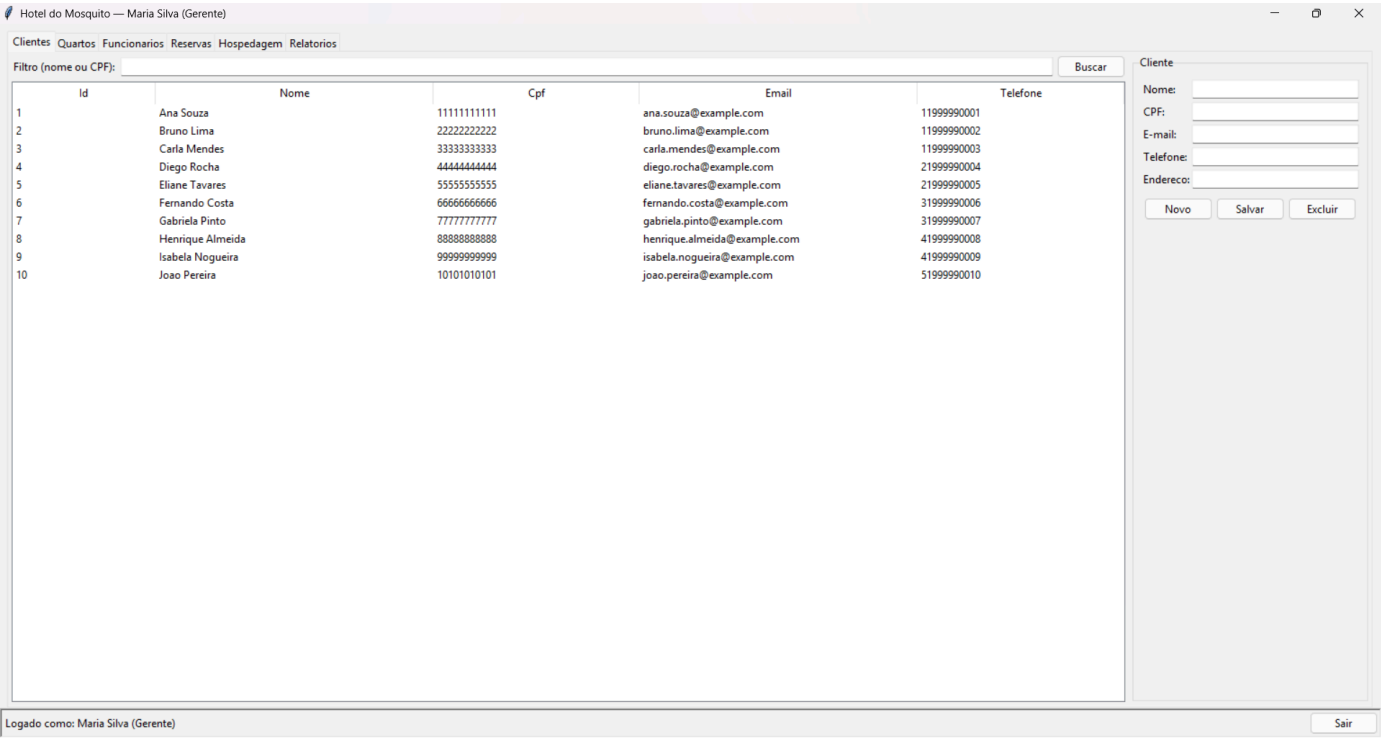


Figura 4 — Tela principal (perfil Recepcionista — 4 abas)

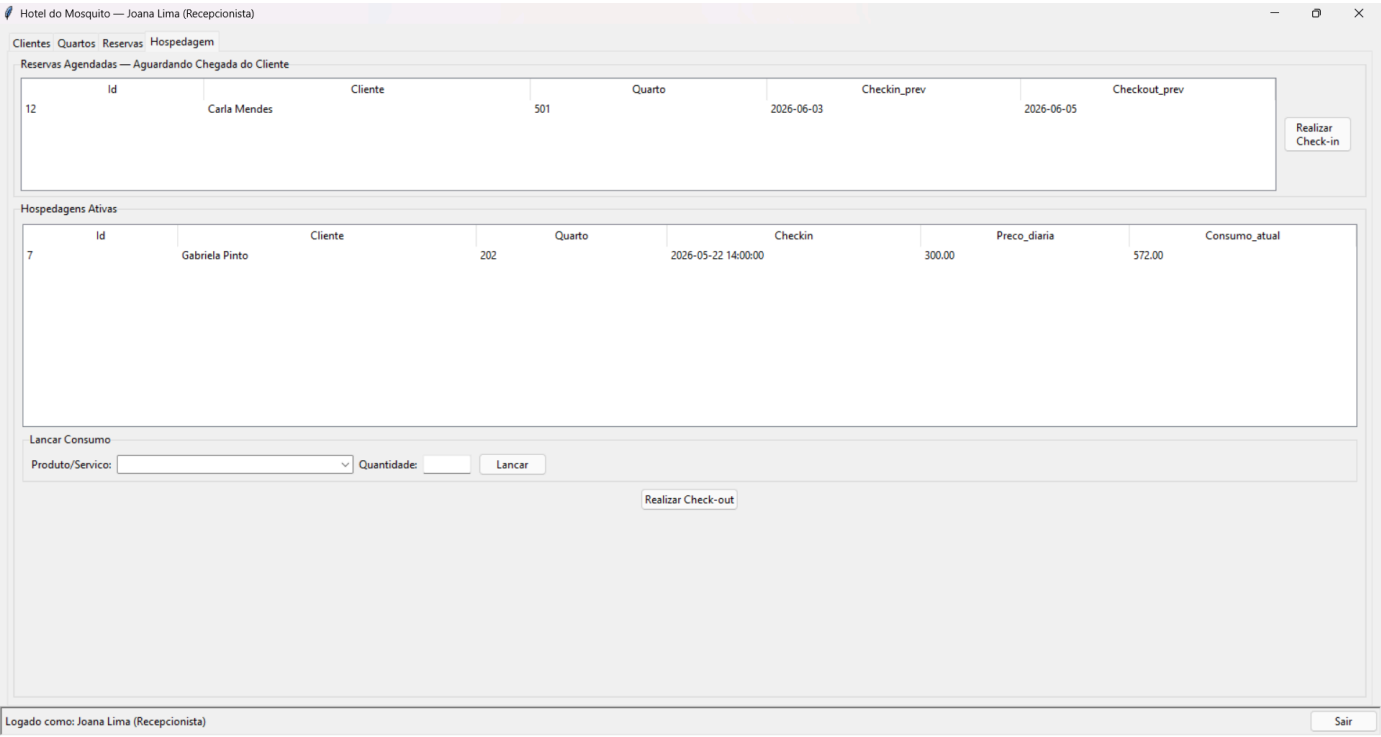


Figura 5 — Aba Hospedagem: reservas agendadas e check-in

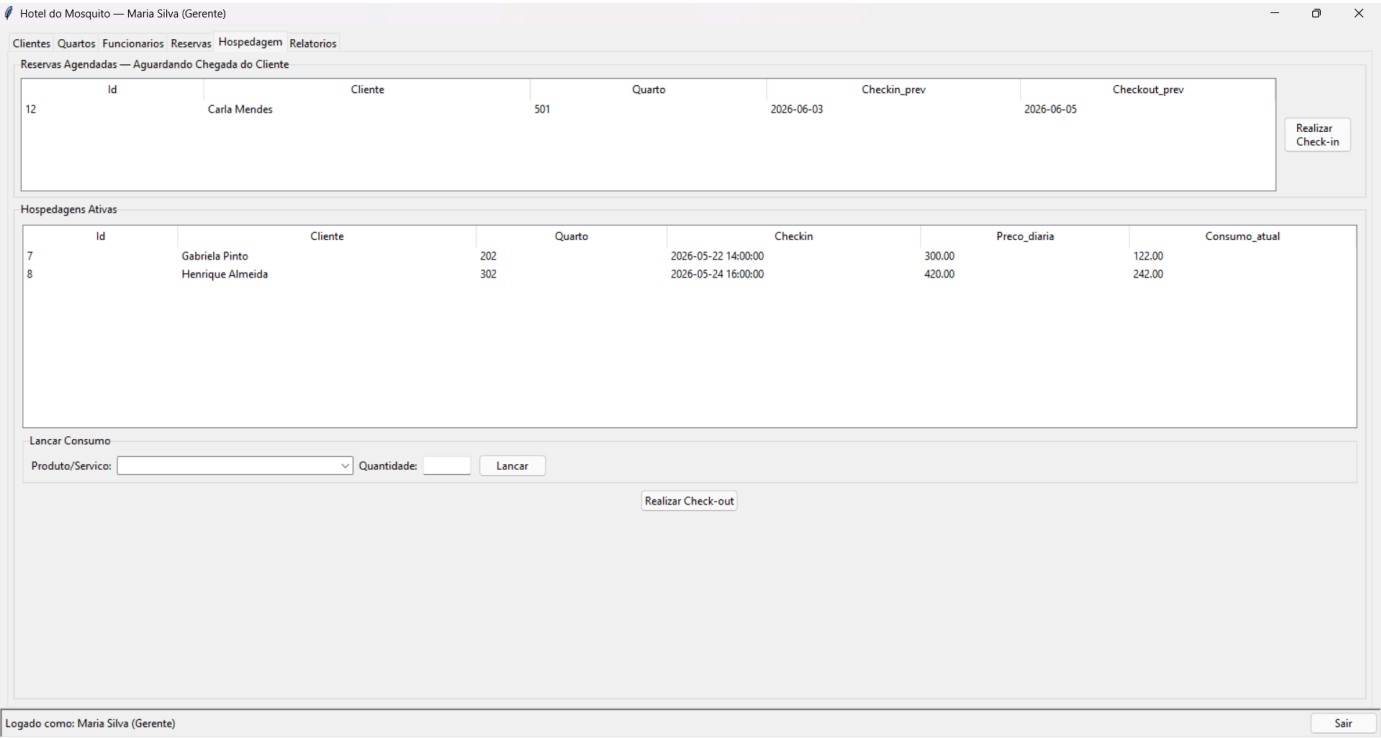


Figura 6 — Aba Hospedagem: lançamento de consumo

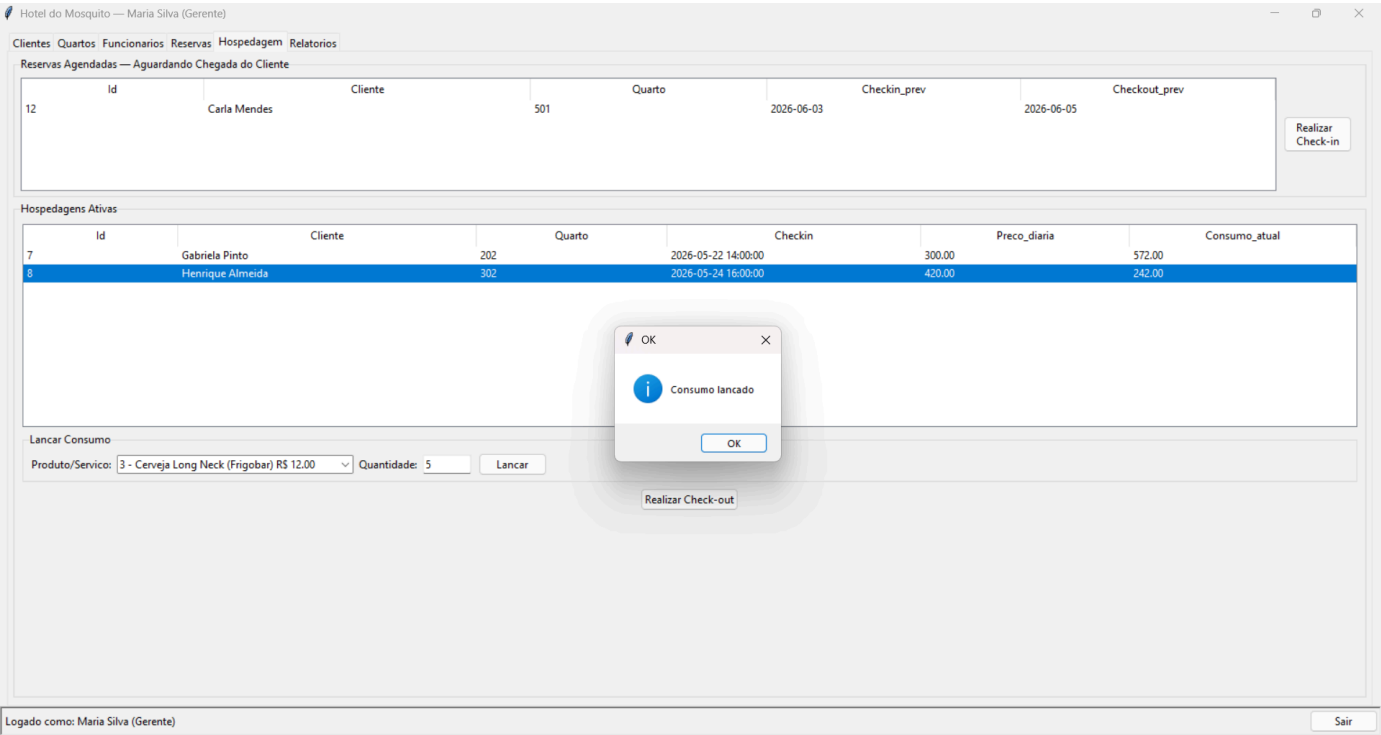


Figura 7 — Modal de check-out com valor total

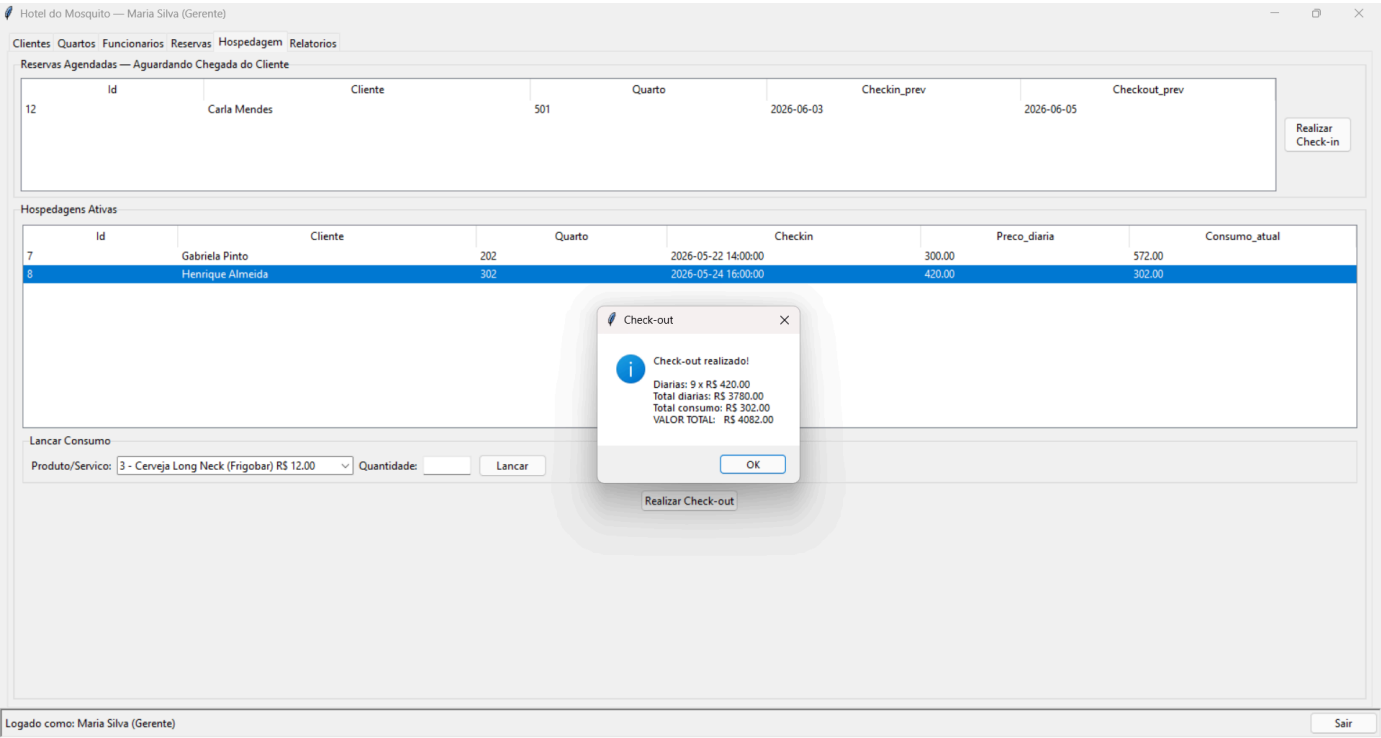


Figura 8 — Aba Quartos (perfil Gerente — formulário completo)

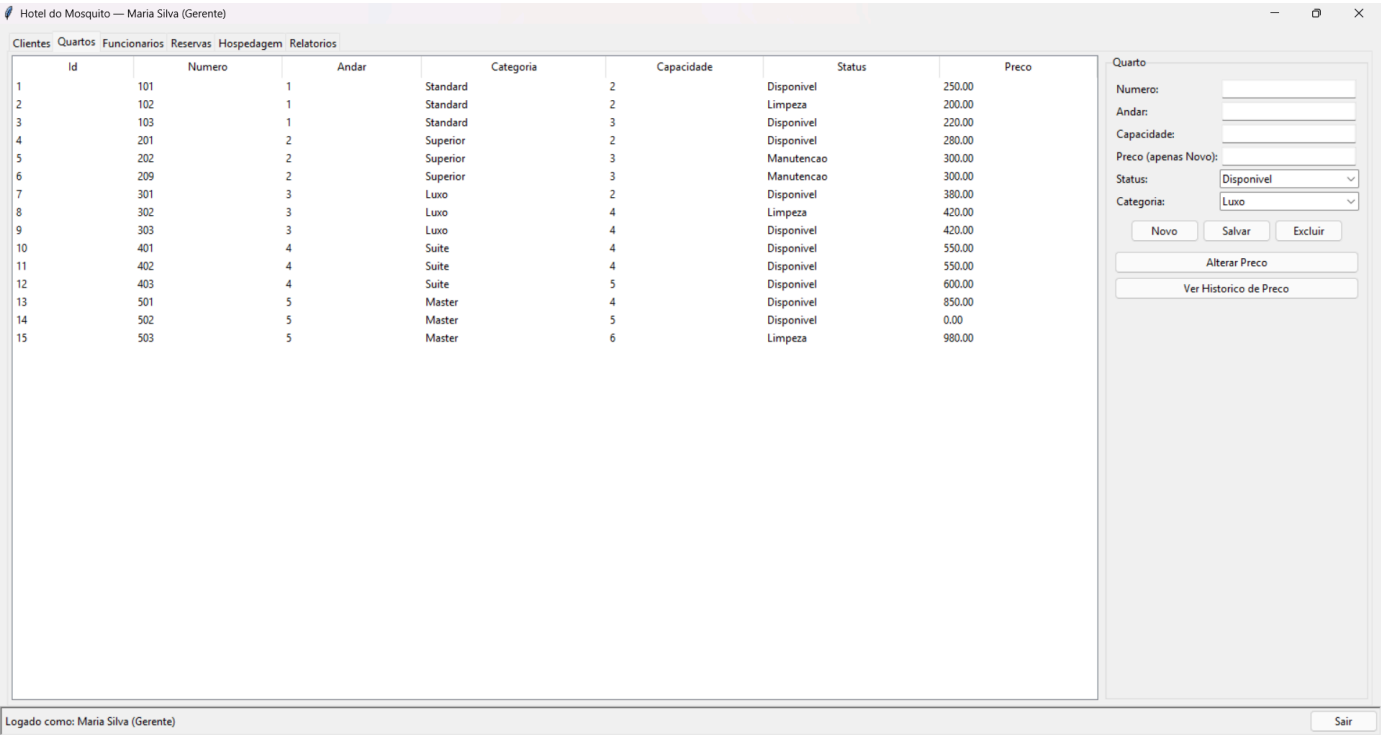


Figura 9 — Aba Quartos (perfil Recepcionista — somente consulta)

Hotel do Mosquito — Joana Lima (Recepcionista)

Clientes Quartos Reservas Hospedagem								
	Id	Numero	Andar	Categoria	Capacidade	Status	Preco	
1	101	1	Standard	2	Disponivel	250.00		
2	102	1	Standard	2	Limpeza	200.00		
3	103	1	Standard	3	Disponivel	220.00		
4	201	2	Superior	2	Disponivel	280.00		
5	202	2	Superior	3	Manutencao	300.00		
6	209	2	Superior	3	Manutencao	300.00		
7	301	3	Luxo	2	Disponivel	380.00		
8	302	3	Luxo	4	Limpeza	420.00		
9	303	3	Luxo	4	Disponivel	420.00		
10	401	4	Suite	4	Disponivel	550.00		
11	402	4	Suite	4	Disponivel	550.00		
12	403	4	Suite	5	Disponivel	600.00		
13	501	5	Master	4	Disponivel	850.00		
14	502	5	Master	5	Disponivel	0.00		
15	503	5	Master	6	Limpeza	980.00		

Logado como: Joana Lima (Recepcionista) Sair

Figura 10 — Aba Relatórios: Faturamento Mensal

Hotel do Mosquito — Maria Silva (Gerente)

Clientes Quartos Funcionarios Reservas Hospedagem Relatorios

Taxa de Ocupacao Top 10 Clientes Top 10 Quartos Faturamento Mensal

Ano: 2026 Mes: 5 Gerar

ano	mes	qtd_checkouts	total_diarias	total_consumos	faturamento_total
2026	5	2	2580.00	101.00	2681.00

Logado como: Maria Silva (Gerente) Sair

10. Casos de teste

Os casos abaixo foram projetados para cobrir os pontos críticos do sistema. A coluna **Observado** deve ser preenchida pelo grupo durante os testes.

#	Cenário	Como reproduzir	Resultado esperado	Observado
1	Login com senha errada	Login gerente1, senha errada	Mensagem "Credenciais invalidas"	Conforme esperado — dialog de erro exibido
2	Login com usuário inexistente	Login fantasma, senha qualquer	Mensagem "Credenciais invalidas"	Conforme esperado — mesma mensagem

#	Cenário	Como reproduzir	Resultado esperado	Observado
				genérica (não revela se login existe)
3	Login correto — Gerente	Login gerente1 , senha senha123	Abre sistema com 6 abas (inclui Funcionários e Relatórios)	Conforme esperado — 6 abas visíveis, aba inicial = Clientes
4	Login correto — Recepcionista	Login recep1 , senha senha123	Abre sistema com 4 abas, aba inicial = Hospedagem	Conforme esperado — abas Funcionários e Relatórios ausentes; aba inicial = Hospedagem
5	Sobreposição de reserva	Reservar quarto já reservado para período coincidente	Mensagem "Período sobreposto"	Conforme esperado — SIGNAL lançado antes do INSERT
6	Check-in em reserva já cancelada	CALL sp_RealizarCheckIn(id_reserva_cancelada, 1)	Mensagem "Reserva nao esta Confirmada"	Conforme esperado
7	Check-out em hospedagem já finalizada	CALL sp_RealizarCheckOut(id_hosp_finalizada, 1)	Mensagem "Hospedagem nao esta Ativa"	Conforme esperado
8	Recepcionista tenta acessar relatório (banco direto)	CALL sp_Relatorio_FaturamentoMensal(3, 2026, id_recep)	Mensagem "Acesso negado: operacao restrita a Gerentes"	Conforme esperado — bloqueio confirmado mesmo via acesso direto
9	Trigger ao alterar preço via procedure	CALL sp_AtualizarPrecoQuarto(1, 350.00, id_gerente)	Nova linha em Historico_Preco ; linha anterior com data_fim_vigencia preenchida	Conforme esperado — 2 linhas para o quarto: uma fechada, uma com data_fim_vigencia = NULL
10	Trigger ao alterar preço diretamente (banco)	UPDATE Quarto SET preco_praticado = 400 WHERE id_quarto = 1	Mesma criação de histórico — trigger é independente do cliente	Conforme esperado — trigger executa independentemente de quem disparou o UPDATE
11	ROLLBACK desfaz trigger	Executar Cenário 2 do 99_demo_transacoes.sql	Historico_Preco volta ao estado anterior ao START TRANSACTION	Conforme esperado — INSERT do trigger desfeito junto com o UPDATE, pois ambos estão na mesma transação
12	Snapshot de preço preserva valor no checkout	Check-in → CALL sp_AtualizarPrecoQuarto (novo preço) → check-out	Fatura usa preco_diaria_aplicado do check-in, não o preço atual	Conforme esperado — valor cobrado reflete o preço do momento do check-in
13	FK RESTRICT impede exclusão de cliente com reservas	CALL sp_Cliente_Delete(id_cliente_com_reservas)	Erro de integridade referencial	Conforme esperado — MySQL rejeita o DELETE com erro de FK
14	Lançar consumo com quantidade zero	CALL sp_LancarConsumo(id_hosp_ativa, 1, 0)	Mensagem "Quantidade deve ser positiva"	Conforme esperado
15	Alterar preço como Recepcionista	Logar como recep1 e acessar aba Quartos	Botão "Alterar Preço" desabilitado; formulário de edição não exibido	Conforme esperado — apenas tabela de consulta visível para Recepcionista

11. Principais dificuldades encontradas

11.1 Encapsulamento total sem SQL cru

A exigência de que nenhuma instrução SQL bruta fosse enviada pela aplicação foi o desafio técnico mais significativo do projeto. A dificuldade não estava em escrever as procedures em si, mas em mapear sistematicamente cada operação da interface gráfica a uma procedure correspondente, sem exceções.

O uso de `cursor.callproc()` do `mysql-connector-python` trouxe uma particularidade: os resultsets de uma procedure não são retornados diretamente, mas precisam ser iterados via `cursor.stored_results()`. Isso exigiu a criação de um helper `call_proc()` centralizado que abstraísse esse comportamento para toda a camada de acesso a dados.

11.2 Detecção de sobreposição de períodos de reserva

A lógica para detectar conflitos de período em reservas de quarto foi um ponto que exigiu atenção especial. A condição intuitiva "os períodos se sobrepõem quando há algum dia em comum" tem uma forma canônica em álgebra de intervalos:

```
dois intervalos [A, B] e [C, D] se sobrepõem quando:  
NOT (B ≤ C OR A ≥ D)
```

Implementar a negação lógica dentro de uma subquery SQL, garantindo que reservas canceladas não fossem consideradas como conflito, foi um exercício de raciocínio que exigiu teste com casos de borda (reserva que começa exatamente quando outra termina, reservas de um único dia, etc.).

11.3 Padrão de snapshot de preço e consistência histórica

Garantir que o valor cobrado em uma hospedagem não mudasse após reajustes de preço do quarto foi um problema de design que demorou para ser compreendido em sua totalidade. A solução com `preco_diaria_aplicado` como coluna na tabela `Hospedagem` — e `preco_unitario_momento` em `Consumo` — parece simples em retrospecto, mas exigiu identificar corretamente o momento exato em que cada snapshot deveria ser capturado (no check-in para diárias; no lançamento para consumos).

11.4 Controle de acesso em duas camadas

Decidir onde implementar cada restrição de acesso (banco vs. UI) foi uma discussão relevante. Chegou-se à conclusão de que a **camada de banco é a única garantia real**: a UI pode ser contornada, mas uma `SIGNAL` no banco não pode. A UI oferece apenas experiência do usuário — um Recepcionista não vê botões que não pode usar. O banco garante que, mesmo que alguém descubra a forma de chamar a procedure diretamente, o bloqueio ainda se aplica.

11.5 Triggers e transações

Uma dúvida inicial foi se os efeitos de um trigger seriam desfeitos por um `ROLLBACK`. A resposta é **sim**: triggers executam dentro da mesma transação do comando que os disparou. Isso foi demonstrado no Cenário 2 do script de demo: o `UPDATE` de preço dispara o trigger que cria um histórico; o `ROLLBACK` subsequente desfaz tanto o `UPDATE` quanto o `INSERT` gerado pelo trigger. Este comportamento é fundamental para a consistência transacional do sistema.

11.6 Configuração do ambiente de desenvolvimento

A configuração do ambiente de desenvolvimento apresentou desafios específicos do ecossistema Windows que merecem registro.

Substituição do MySQL Workbench pelo Docker: o enunciado previa o uso do MySQL Workbench como ferramenta de administração local. A opção por Docker Compose para containerizar o MySQL 8.0 mostrou-se mais prática para garantir paridade de ambiente entre os cinco integrantes, mas exigiu atenção ao mapeamento de portas (host 3307 → container 3306) para evitar conflito com possíveis instalações locais de MySQL na porta padrão 3306. O Docker Desktop no Windows requer que o mecanismo Linux esteja completamente inicializado antes de qualquer operação — na primeira inicialização, comandos `docker exec` falhavam com erro de pipe até o engine ficar pronto.

PATH do Git no PowerShell: após a instalação do Git for Windows, o terminal PowerShell já aberto não reconhecia o comando `git` porque o PATH é carregado no início da sessão. Foi necessário reabrir o terminal ou adicionar manualmente o caminho `C:\Program Files\Git\cmd` ao PATH da sessão corrente.

Bug de inicialização do Tkinter: durante o desenvolvimento, identificou-se que `tkinter.messagebox.showerror()` não pode ser chamada antes de `tk.Tk()` ser instanciado — o que causava falha silenciosa nos caminhos de erro (`config.ini` não encontrado, falha de conexão). A correção exigiu reorganizar a ordem de inicialização em `app/main.py`, criando a janela raiz e ocultando-a imediatamente com `root.withdraw()` antes de qualquer chamada a `messagebox`.

Flag `CLIENT_FOUND_ROWS` do driver MySQL: o `mysql-connector-python`, por padrão, configura `ROW_COUNT()` para retornar apenas linhas *modificadas* (não as *encontradas*) após um `UPDATE`. Isso fazia com que atualizações com os mesmos valores retornassem 0 e as procedures lançassem erroneamente "registro não encontrado". A solução foi passar `client_flags=[ClientFlag.FOUND_ROWS]` na conexão, alinhando o comportamento do driver ao esperado pelas procedures.

Encoding de linha (CRLF × LF): o Git no Windows, por padrão, converte terminações de linha LF para CRLF nos arquivos do repositório. Isso gerou avisos em todos os arquivos no primeiro commit. Não causa problema funcional, mas é importante que todos os integrantes usem as mesmas configurações de `core.autocrlf` para evitar diffs espúrios.

*Relatório gerado com base na implementação completa do sistema Hotel do Mosquito.
Código-fonte disponível em: <https://github.com/Matheus-PC-Silva/hotel-mosquito-labdb>*